

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №5

Реализация межсервисного взаимодействия посредством очередей сообщений

Выполнил:
Мальцев И. И.
БР 1.2

Проверил:
Добряков Д. И.

2026 г.

Задача

В рамках домашнего задания необходимо подключить и настроить брокер сообщений RabbitMQ, а также реализовать межсервисное взаимодействие через очередь сообщений. Работа выполнялась для backend-приложения сайта поиска работы.

Основная цель работы - показать, что отдельные части системы могут обмениваться событиями без прямого синхронного вызова друг друга. В качестве сценария было выбрано взаимодействие между сервисом откликов и сервисом уведомлений.

Ход работы

Для реализации межсервисного обмена был выбран RabbitMQ. В проект был добавлен docker-compose.yaml с контейнером RabbitMQ, после чего в коде был реализован отдельный слой platform/mq для работы с очередями сообщений.

Для обмена событиями был создан exchange job_search.events. Сообщение имеет общий формат EventMessage: тип события, дата создания и полезная нагрузка в JSON. Такой формат позволяет разным сервисам обрабатывать события одинаковым способом, но при этом хранить внутри payload данные конкретного действия.

- applicant-service публикует события, связанные с откликами соискателей;
- notification-service подписывается на сообщения и обрабатывает их;
- gateway оставлен точкой входа для HTTP API;
- auth-service, catalog-service и employer-service продолжают отвечать за свои части предметной области.

В работе использовались два основных события: application.created и application.status_changed. Первое событие создается при отклике соискателя на вакансию. Второе событие создается при изменении работодателем статуса отклика. В реальном приложении notification-service мог бы на основе этих событий отправлять уведомление пользователю.

Сценарий межсервисного взаимодействия

Действие	Сервис-отправитель	Событие	Сервис-получатель
Создание отклика	applicant-service	application.created	notification-service

Изменение статуса отклика	applicant-service / employer flow	application.status_changed	notification-service
---------------------------	-----------------------------------	----------------------------	----------------------

Для проверки была запущена команда `go run ./cmd/job-search-microservices`. В логах видно, что контейнер RabbitMQ находится в состоянии Running, exchange объявлен, а сервисы поднялись на отдельных портах. Это подтверждает, что сервисы работают как независимые части приложения и готовы обмениваться событиями через брокер.

Проверка работы

После запуска сценария проверка показала, что взаимодействие через RabbitMQ проходит успешно. При создании отклика и изменении его статуса формируется событие, которое публикуется в exchange. Notification-service получает сообщение и выводит информацию об обработке в логах.

Также были выполнены автоматические проверки проекта. Команда `go test ./...` проходит успешно. Кроме этого, был проверен `docker compose config` и тестовый сценарий через RabbitMQ. В результате ошибок при запуске и обработке сообщений не возникло.

Вывод

В ходе работы было реализовано межсервисное взаимодействие через RabbitMQ. Для приложения сайта поиска работы был добавлен общий слой работы с очередью сообщений, настроен exchange `job_search.events` и реализована отправка событий `application.created` и `application.status_changed`.

Такой подход уменьшает связанность между сервисами: сервис откликов не вызывает сервис уведомлений напрямую, а только публикует событие. Получатель сам обрабатывает сообщение из очереди. Это делает архитектуру более гибкой и ближе к микросервисному подходу.